DSE Control Plane — Metrics Reference

Every number on Admin Home and Analytics: what it means, where it is computed, and whether it works today.
Each component below is a screenshot of the **real console**, rendered from `control-plane/admin-ui` against the live production data on 2026-09-01.

1. Where the numbers come from

Every figure on both screens is derived from three tables. Nothing is computed in the browser, and nothing is estimated unless this document says so.

```
DSE Postgres Console (SQLite) UI
audit_log ███████
███ console_rm.runs_view ████████ runs ████████
model_call_ledger ██ ███ analytics.ts ██ /analytics/home
work_items ███ console_rm.work_items_view ███ work_items ███ /analytics
audit_log ████ console_rm.timeline_events ██ events █████████
```

A “run” is one agent turn — not one work item, not one HTTP call. Each row in `runs_view` carries the stage that produced it (planner, coder, tester, reviewer), its token counts and its cost in USD.

Source	Live content (2026-09-01)
runs_view	1,007 rows · $860.28 · 12,048,429 tokens · 2026-07-24 → 2026-09-01
by stage	coder 502 ($851.61) · tester 267 ($4.51) · planner 204 ($4.02) · reviewer 34 ($0.14)
work_items_view	184 rows — blocked 100 · failed 77 · pr_ready 7
console_rm.baselines	0 rows — not configured
status = done	never occurred (0 events)

Two of those lines govern most of what follows. **No work item has ever reached** `done`, and **no ROI baseline is configured**. Between them they explain every “—” on the dashboard.

2. Admin Home — Cost & Impact

2.1 LLM spend, last 30 days

The real card, live data.

Computation: sum of `cost_usd` over every run whose `started_at` falls in the last 30 days. The sparkline buckets the same runs into 8 equal slices of that window. `analytics.ts:109`

Delta "+1627% mo": percentage change against the *previous* 30 days, with the tone inverted so that rising cost reads red. `analytics.ts:37`

Status: works, but the delta is not usable. The figure is arithmetically correct and practically meaningless — the previous 30-day window contained almost no spend, so any activity at all produces a four-digit percentage. Consider suppressing the delta when the base period is below a threshold.

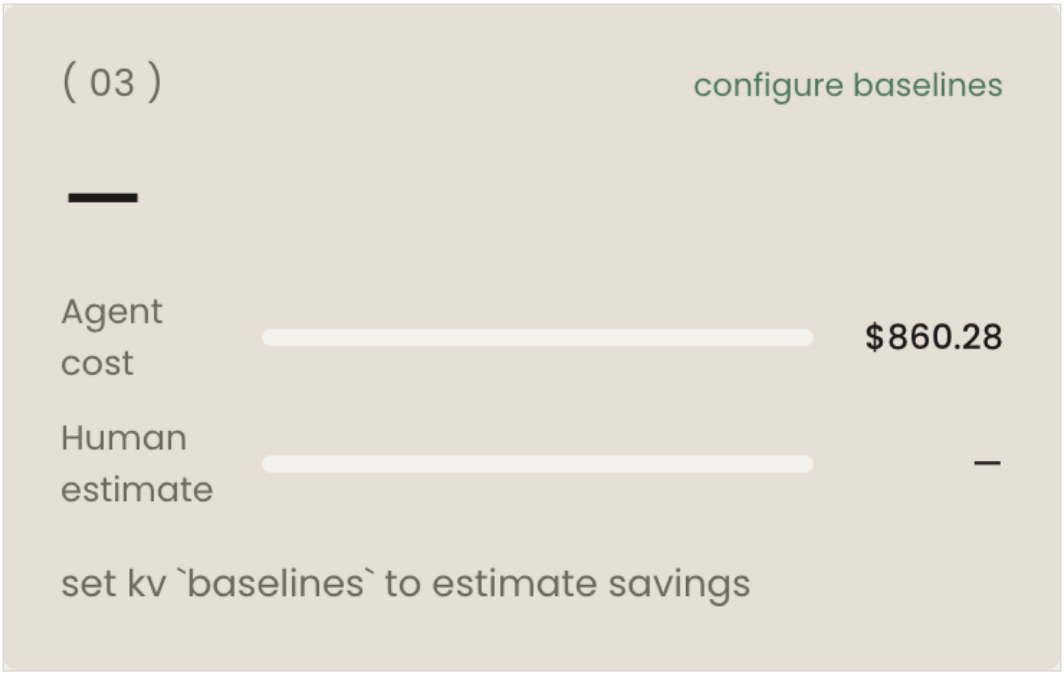
2.2 Tokens, last 7 days

The real card, live data.

Computation: sum of `tokens_in + tokens_out` over runs started in the last 7 days, compared with the 7 days before it. The bars are the same runs in 7 daily buckets.

Status: works. The empty bars in the middle are real — they are days with no runs.

2.3 Savings vs human estimate



The real card. Both rows are empty because no baseline is configured.

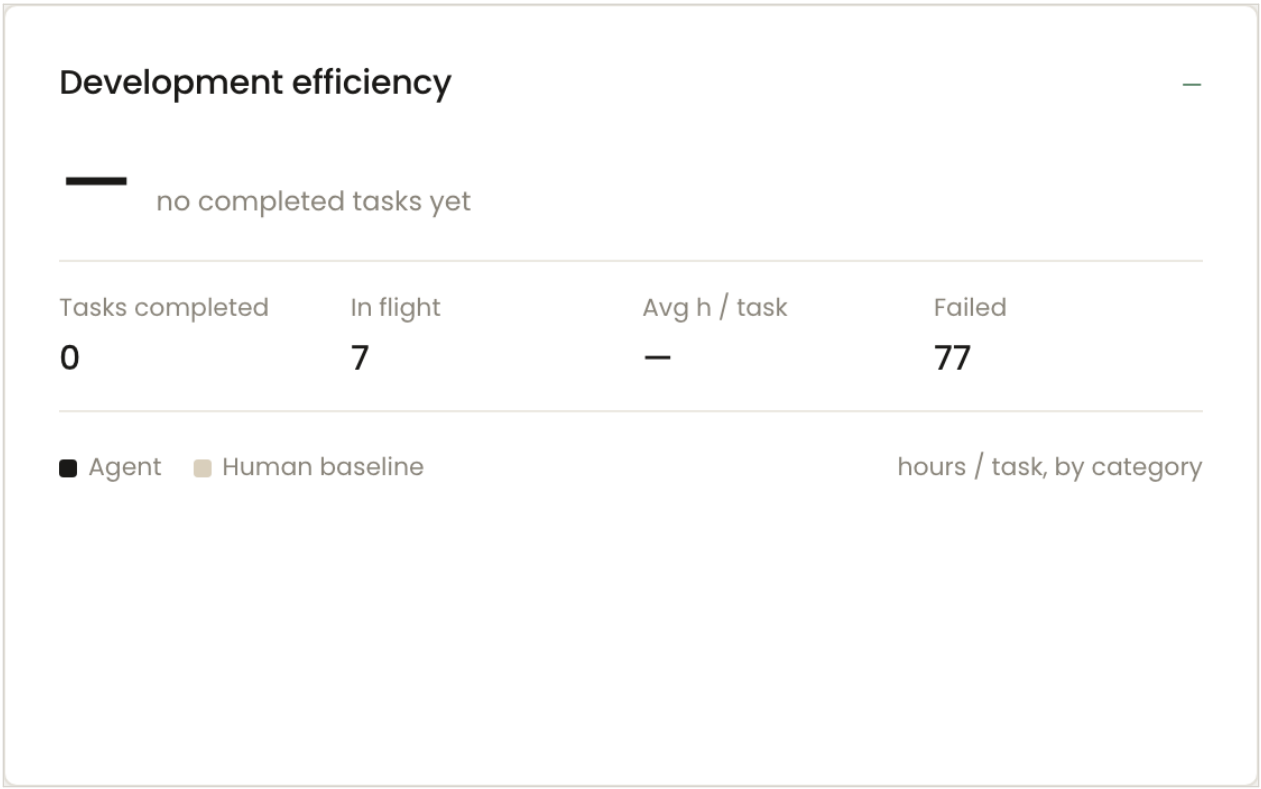
Computation: `completed_tasks × baseline_hours × hourly_rate − agent_cost`. It needs a `baselines` record (hourly rate and expected human hours per task), which has never been set.

Agent cost ($860.28) is the sum of `cost_usd` over **all runs ever** — not the 30-day window used by the card at the top of the same row ($813.20). `analytics.ts:125`

Status: correctly blank, but the two cost figures mislead. Rendering "—" without a baseline is the right behaviour — the alternative is inventing a human cost. The defect is that two numbers on one screen look like they should agree and never will, because one is all-time and the other is 30 days, and neither says so.

3. Admin Home — Efficiency & Quality

3.1 Development efficiency

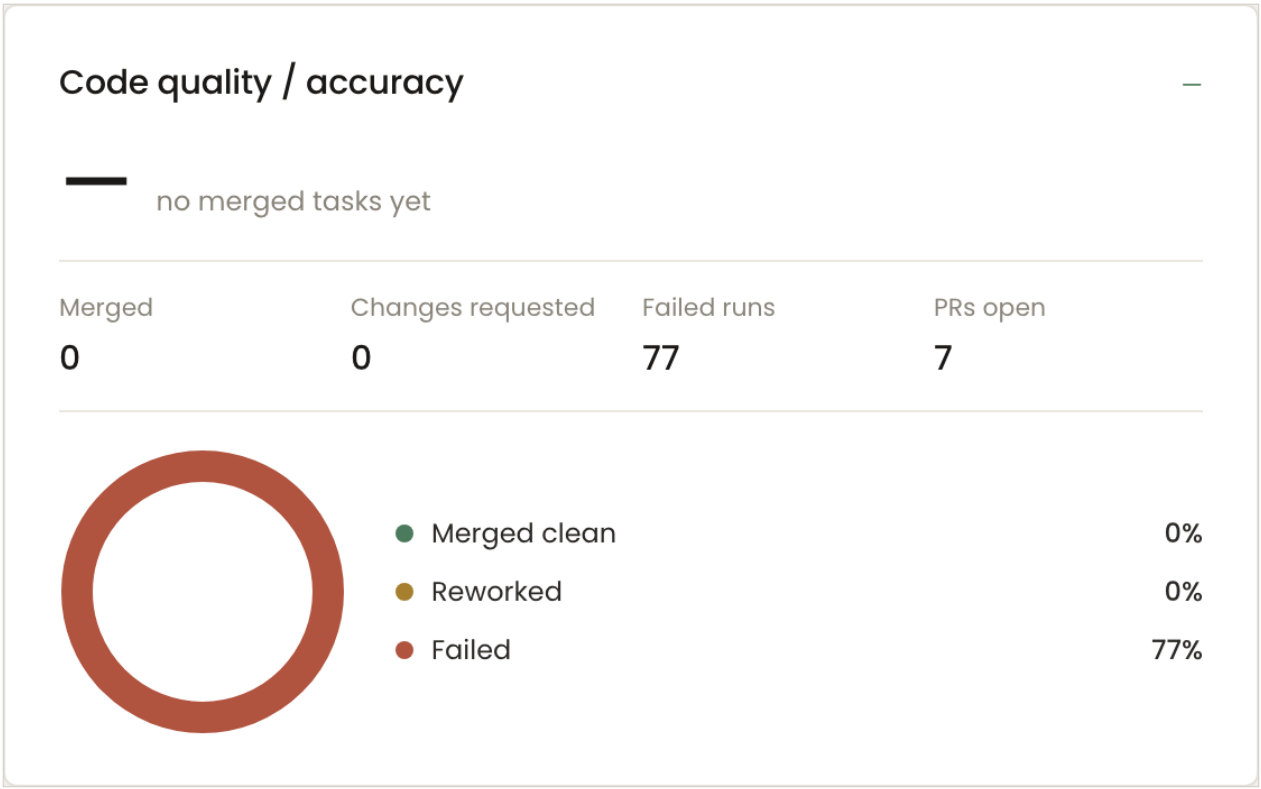


The real card. Five of its seven figures cannot populate today.

Figure	How it is computed	Status
Headline (avg h)	Mean of `done_at − created_at` across completed items. `done_at` is the timeline event whose message starts with "→ done".	**NO DATA**
Tasks completed	Count of items with status `done`.	**NO DATA**
In flight	Count of {queued, running, pr_ready, review_feedback}. The 7 shown are the `pr_ready` items.	**WORKS**
Avg h / task	Same mean as the headline.	**NO DATA**
Failed	Count of items with status `failed`.	**WORKS**
hours / task by category	Per repository: mean agent hours against the configured human baseline.	**NO DATA**

These are gated on a status the product cannot currently reach. An item becomes `done` only after a human merges its pull request. The DSE has opened 14 pull requests in its history and none were merged, so this card has never had an input. Not a calculation bug — a definition question, addressed in §6.

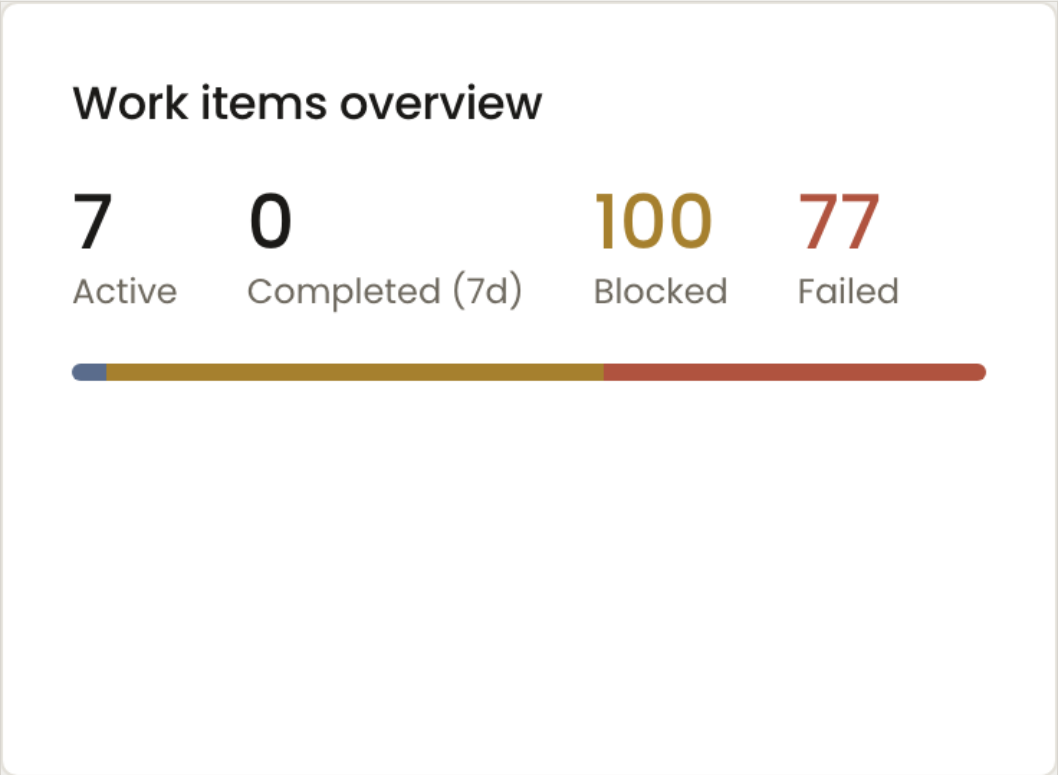
3.2 Code quality / accuracy



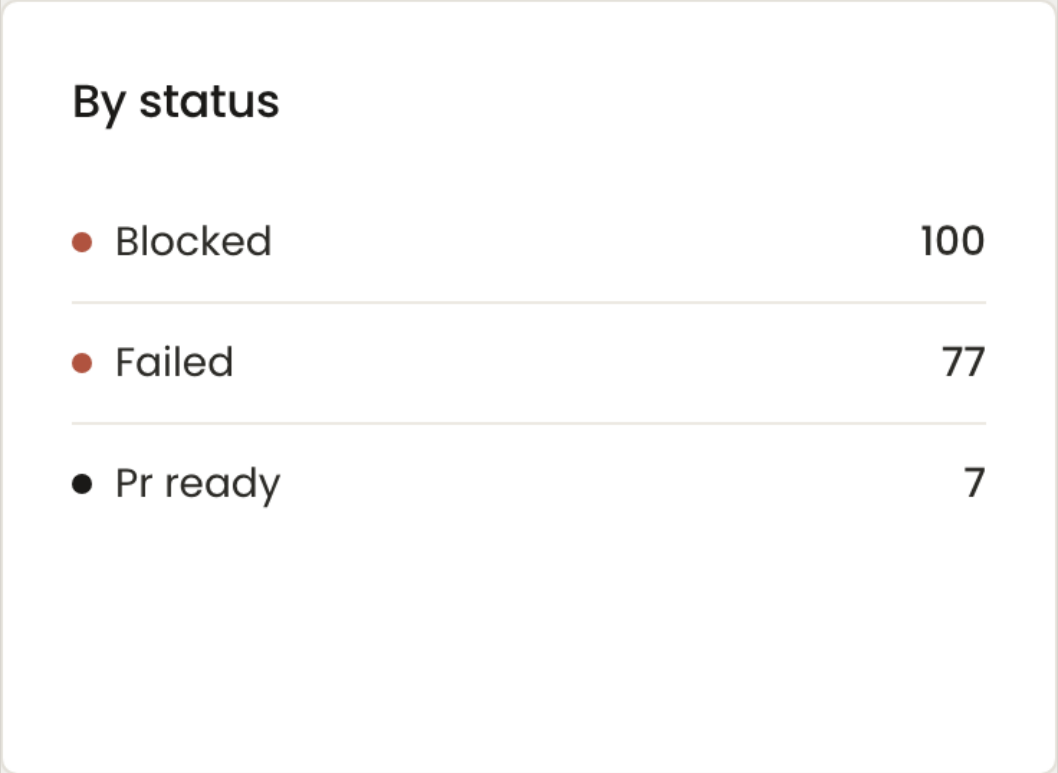
The real card. Note the legend: “Failed 77%”.

Figure	How it is computed	Status
First-pass rate	`(completed − reworked) / completed`, where reworked means the timeline contains a review-feedback event.	NO DATA
Merged	Count of status `done` — a proxy. No GitHub merge state is read.	MISLEADING
Changes requested	Completed items whose timeline shows review rework.	NO DATA
Failed runs	**Counts work items, not runs.** It is the same 77 shown as “Failed” on the efficiency card. `analytics.ts:205`	BROKEN
PRs open	Count of {pr_ready, review_feedback}.	WORKS
Donut legend	**Renders a count with a percent sign.** “Failed 77%” is 77 work items; the real share is 42%. `admin-ui/src/app/page.tsx:316`	BROKEN

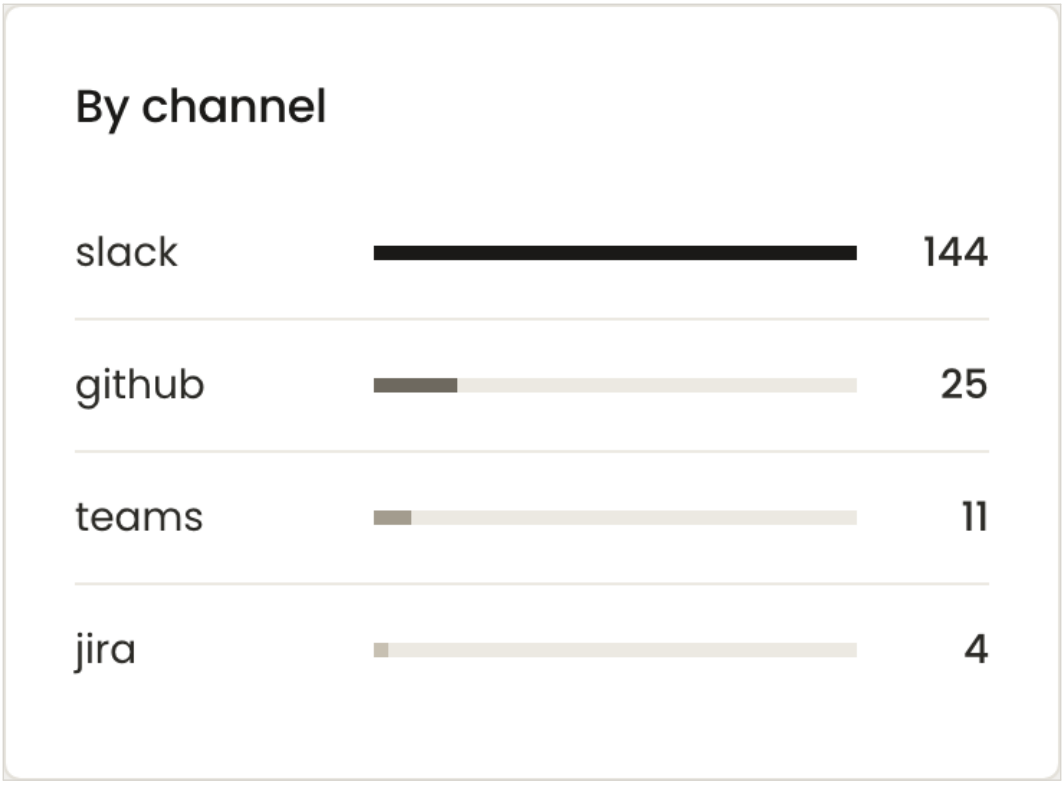
4. Admin Home — Work item activity



Plain counts by status — the most trustworthy numbers on the page.



The same counts as a list. “Blocked” is the largest bucket on the dashboard.



Work items grouped by the intake channel that created them.

Status: all three work. They are direct counts over a table the projector maintains. Worth noting that **blocked (100) is the biggest bucket anywhere on the dashboard and nothing explains why** — there is no breakdown of blocking reason on either screen. That is a missing metric, not a broken one.

5. Analytics

The three tabs read the same two tables, restricted to the selected range and filters. The range applies to `started_at` for runs and to `created_at`/`updated_at` for work items, so a long-running item can appear in a window its runs do not.

5.1 Filters

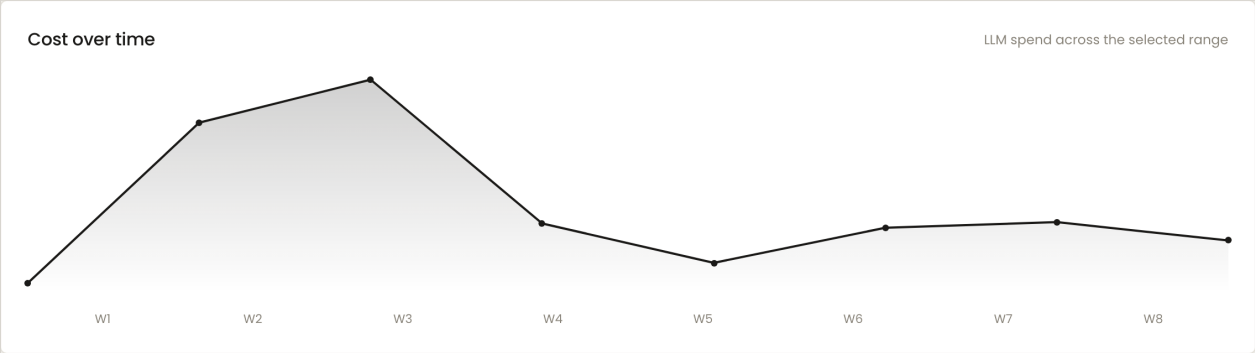
The real filter bar.

Filter	Behaviour	Status
Date range	7d / 30d / 90d / QTD. QTD starts at the first day of the current calendar quarter.	WORKS
Repository	Filters work items by repo, then keeps only the runs belonging to those items.	WORKS
Agent	Filters by the runtime assigned to the item.	WORKS
Project · Task type	**Inert.** Both are hard-coded to a single option and filter nothing.	BROKEN

5.2 Cost tab

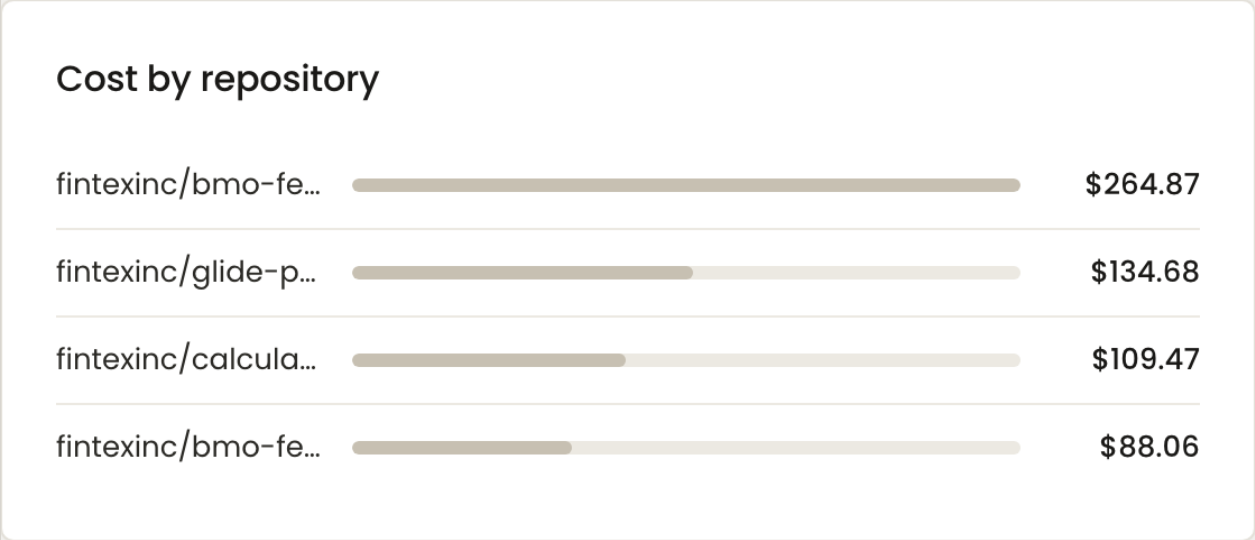
The four summary cards, live.

Figure	How it is computed	Status
Total cost	Sum of `cost_usd` over runs in range.	**WORKS**
Tokens	Sum of `tokens_in + tokens_out` over the same runs.	**WORKS**
Runs	Row count of those runs — agent turns, not work items.	**WORKS**
Cost / completed task	`total_cost / completed_items`. "—" while nothing is completed, which is the current state.	**NO DATA**



Cost over time — the runs in range, bucketed into 8 equal slices.

Status: works. The shape is real: the peak at W3 is the day the fix loop ran hardest.



Cost by repository.

Computation: each run is attributed to its work item's repository, then summed. Runs whose item falls outside the filtered set land in "unknown". **Status: works.**

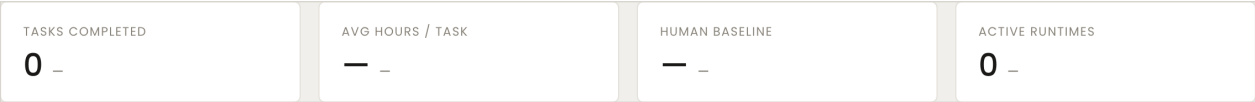
Cost by model — a single bar, and the wrong one.

Status: broken. The chart sums cost by the model name recorded on each run, and every stage is recorded as `anthropic/claude-haiku` — including the Coder, whose configured model is `anthropic/claude`. The cost per run proves they are not the same model:

```
coder 467 runs $822.21 = $1.76 / run labelled haiku
planner 204 runs $4.02 = $0.02 / run labelled haiku
tester 267 runs $4.51 = $0.02 / run labelled haiku
```

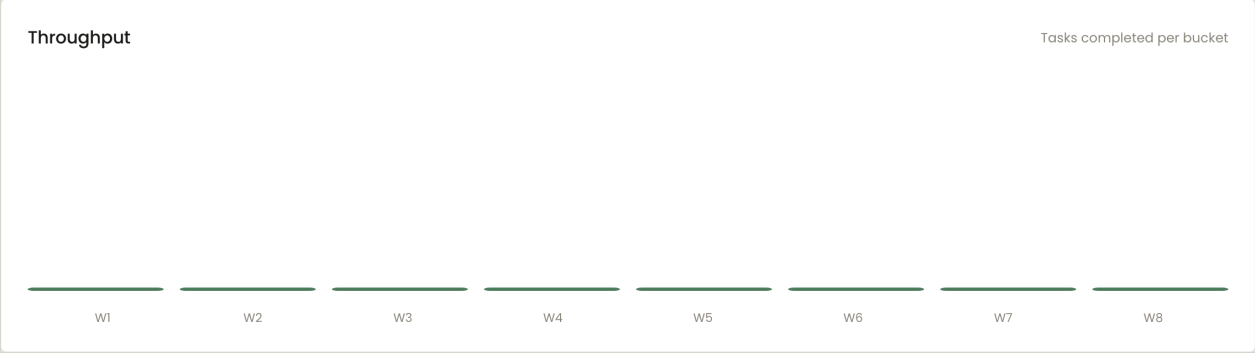
A ninety-fold difference in cost per run between calls to the same model is not plausible. The label originates in `model_call_ledger` at metering time; the projector copies it verbatim. A further 35 Coder runs ($29.40) carry no model at all. **This chart exists to support choosing a cheaper model, and it would actively mislead that decision.**

5.3 Efficiency tab

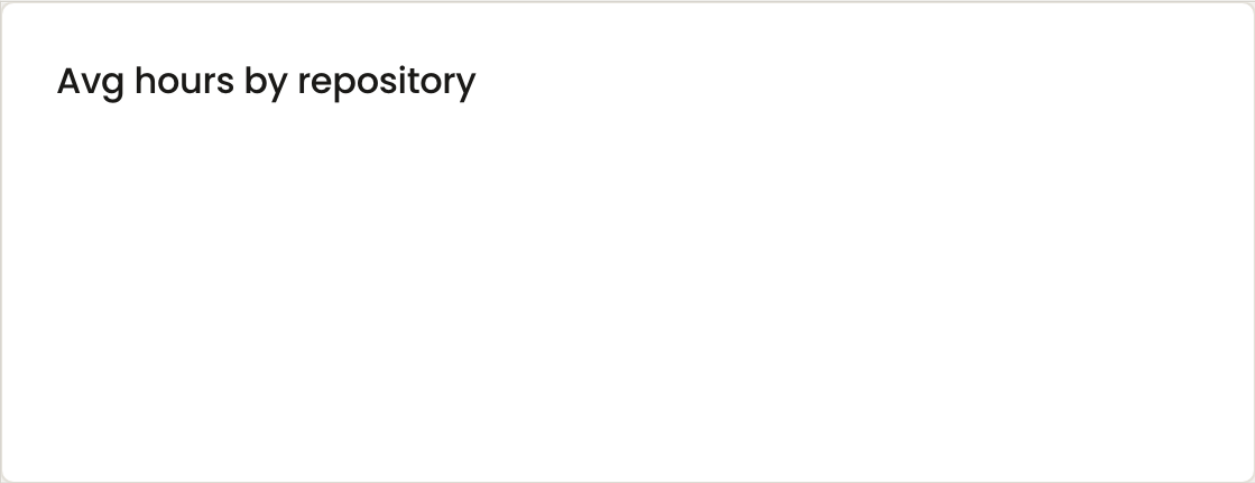


The four summary cards. Three of them cannot populate.

Figure	How it is computed	Status
Tasks completed	Count of status `done` in range.	NO DATA
Avg hours / task	Mean wall-clock from item creation to its "→ done" event.	NO DATA
Human baseline	Read from the configured baseline; "—" when unset.	NO DATA
Active runtimes	Distinct non-null runtimes across items in range.	WORKS



Throughput — completed items per bucket. Empty because nothing completes.



“Avg hours by repository”.

Status: broken. The title promises an average; the code aggregates with a **sum** and never divides. `analytics.ts:321-323`. With at most one completed item per repository the two would agree, which is why it has never been noticed.

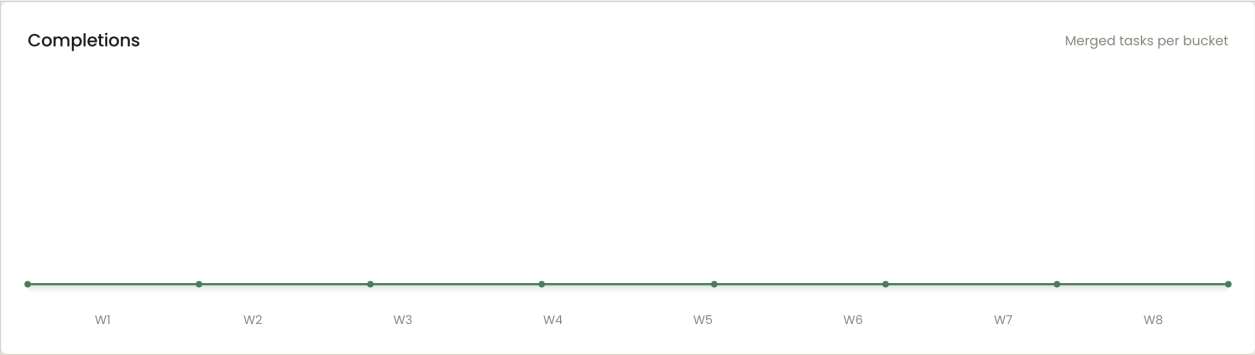
Work items by status — done / in progress / failed.

Status: works, with the same count-as-percent legend defect as the Admin Home donut.

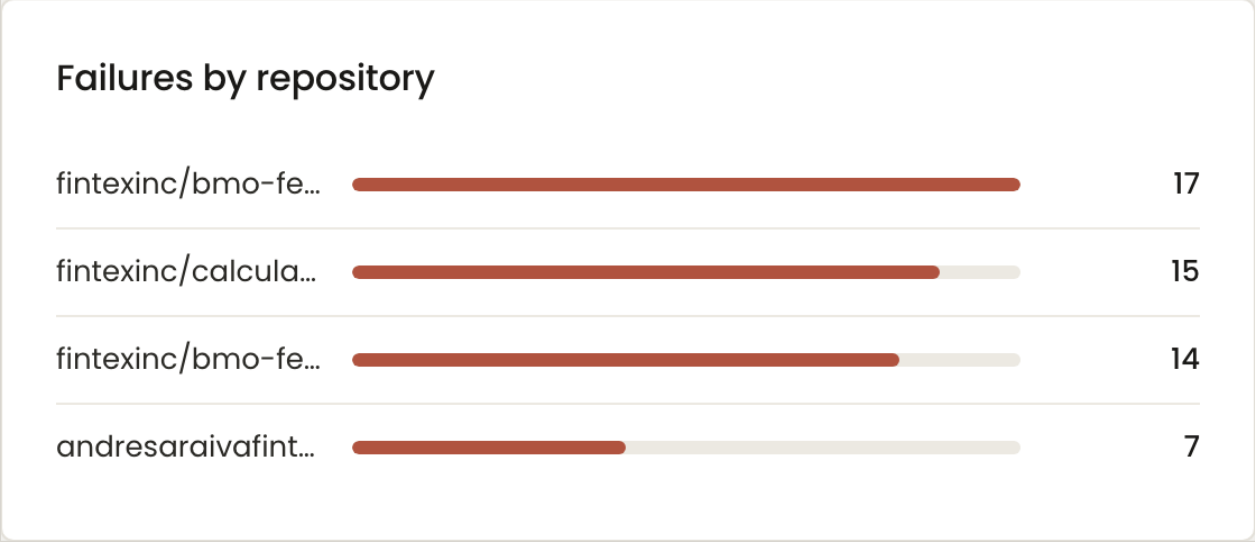
5.4 Quality tab

The four summary cards.

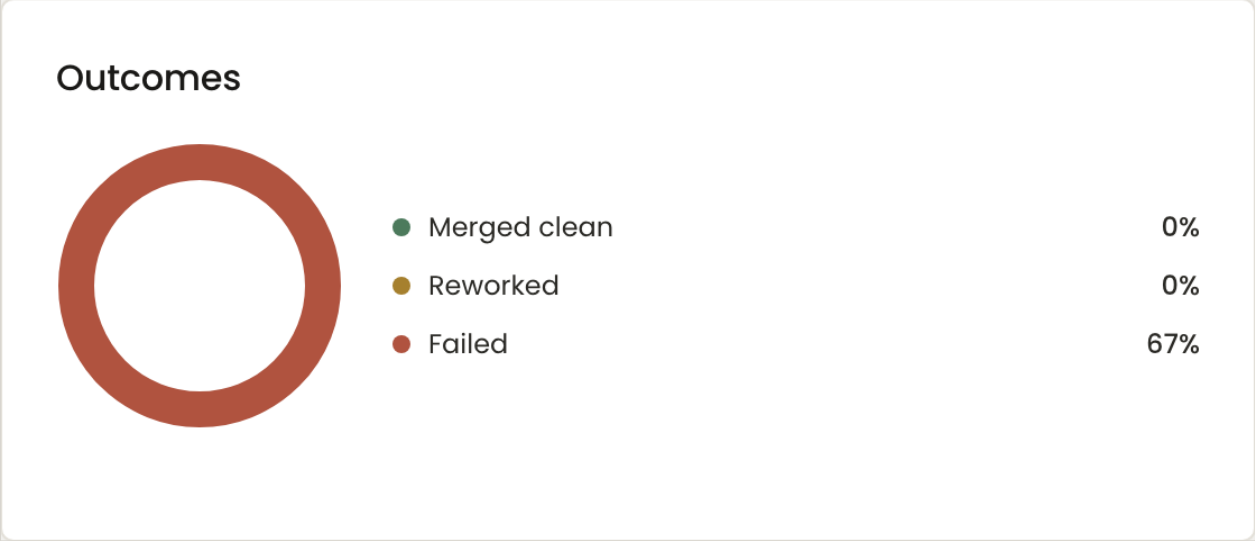
Figure	How it is computed	Status
First-pass rate	Completed items with no review-rework event, over completed items.	NO DATA
Changes requested	Completed items that were reworked.	NO DATA
Failed	Count of failed items in range. Correctly labelled here, unlike Admin Home.	WORKS
PRs open	Count of {pr_ready, review_feedback}.	WORKS



Completions per bucket.



Failures by repository — real counts.



Outcomes donut, with the same percent defect.

6. The structural question: nothing is ever “done”

Eleven of the figures above report “no data”, and they share one cause. Every efficiency and quality metric on both screens is gated on work items reaching status `done`, and `done` is assigned only after a human merges the pull request. In the platform's entire history no item has ever reached it.

The metrics are not wrong. The definition of “completed” is one the operator cannot satisfy by using the product. The result is a dashboard whose two headline quality panels have never displayed a value, while the

platform has in fact produced 14 pull requests.

Option	What changes	Trade-off
Leave as is	Nothing. The panels populate the first time somebody merges a DSE pull request.	Honest, and useless until then. The dashboard cannot show progress on the work being done.
Add a second milestone	Track “delivered” (PR opened and green) alongside “done” (merged). Efficiency measures creation-to-PR; quality keeps measuring merges.	Two definitions to explain, but each answers a real question: how fast the agent works, and how often its work is accepted.
Redefine done as PR-opened	One line in the status mapping.	Cheapest, and it destroys the quality panel: “merged without rework” means nothing if nothing was merged. **Not recommended.**

Recommendation: the second. It is the only option that lets the dashboard describe what the platform actually does today — open pull requests, and not get them merged — instead of showing a blank card that reads as “no activity”.

7. Summary and suggested order of work

#	Item	Action	Effort
1	Donut legend renders counts as percentages (both screens)	Fix	1 line × 2 files
2	Cost by model attributed to the wrong model	Fix or hide	Metering change
3	“Failed runs” label counts work items	Rename	1 line
4	Agent cost is all-time beside a 30-day figure	Label or rescope	1 line
5	“Avg hours by repository” sums instead of averaging	Fix	1 line
6	Project / Task type filters do nothing	Remove	Delete two controls
7	“+1627%” delta from a near-zero base	Suppress below a floor	1 condition
8	Nothing ever reaches “done”	Product decision — §6	Design first
9	“Blocked” is the biggest bucket and unexplained	Add a reason breakdown	New query

Items 1, 3, 4, 5, 6 and 7 are single-line corrections to display logic and carry no risk. Item 2 changes what is recorded and should be verified against a known model call before it is trusted. Item 8 is not an engineering task — it is a decision about what the product counts as finished, and every empty panel above is waiting on it.